UNIX INTERNALS - ASSIGNMENT-3

SIVA.S
2023115113

1. BLOCK ADDRESSING:

given:

* Disk block size = 1024 bytes.
* No. of direct address blocks = 10.
* single Indirect = 7000; double = 8000, blocksize = 4 bytes.
* No. of blocks in 1 indirect block = $\frac{1024}{4} = 256$
* Data addressable by one single-indirect block is = $256 \times 1024 = 262,144$ bytes.

Inode block Information:

Block Type	Block No.
0	500
1	501
2	502
3	503
4	504
5	505
6	506
7	507
8	508
9	509
Single Indirect	7000
Double Indirect	8000

(a) Byte offset

$\Rightarrow$ Logical block = $\left\lfloor \frac{9000}{1024} \right\rfloor = 8$

Byte offset within the block = $9000 - (8 \times 1024)$
$= 808.$

Block Access sequence.

Inode $\rightarrow$ Direct[8] $\rightarrow$ Block 508
reads byte 808.

(b) Byte offset = 200,000.

Logical block = $\left\lfloor \frac{200000}{1024} \right\rfloor = 195$

∴ logical block 195 lies in the single-indirect region.

⇒ Index of single-Indirect ⇒

$195 - 10 = 185$

∴ entry = 7000[185].

Find the byte offset within the final Data Block = $200000 - (195 \times 1024)$

$= 200000 - 199680 = 320$.

(c) Byte offset = 300,000

Logical block = $\left\lfloor \frac{300000}{1024} \right\rfloor = 292$

direct → 10 blocks

single Indirect → 256 blocks. So, $292 > 10 + 256$

∴ It is in double-Indirect.

position in double-Indirect = $292 - 266 = 26$.

outer Index = $\left\lfloor \frac{26}{256} \right\rfloor = 0$.

Inner Index = $26 \bmod 256 = 26$.

Byte offset within the final Data Block.

$= 300000 - (292 \times 1024)$

$= 300000 - 299008$

$= 992$

2. Bach's iget Algorithm

When a process requests inode 75, which is already present in the hash queue but locked by another process, the following steps occur.

1. Hash Queue search : Kernel searches the appropriate hash queue and finds inode 75.
2. Check Lock : It finds that inode 75 is locked.
3. Sleep : The requesting process sleeps and waits for the inode to be unlocked.
4. Wake-up: When the other process releases the inode lock, the waiting process is awakened.
5. Recheck : The process searches / rechecks the inode again to ensure it is still present and unlocked.
6. Reference Count : The inode's reference count is incremented.
7. Return : The inode is locked for the requesting process and returned by iget.

Sequence:

search → Find inode 75 → Locked → sleep → wakeup → Recheck → Increment reference count → Lock and return.

Why recheck?

Rechecking is necessary because the inode's state may have changed while the process was sleeping. This prevents race conditions during concurrent inode access.